ICD-10-CM Clinical Ontology Module — Design (Open RFC)

Module: `hdh.modules.icd10cm` · **Status:** open RFC (design, not yet built) · **Date:** 2026-08-08

Contributors

Name	Role	Contribution
Ajmal Mahmood	Author / Architect	Reference architecture, knowledge-graph framework, this design

To be added as a contributor: submit design feedback or implementation work via a PR or issue on this document, and add yourself to the table in the same change.

A design for a **lookup-optimized clinical ontology module** built around ICD-10-CM: the full ~74,000-code catalog loaded as a knowledge graph — hierarchy, laterality, episode-of-care, and coding-rule relationships as first-class edges — delivered as an **hdh schema-registry module**, running on **PostgreSQL + Redis dependency containers** started by `just deps`, with a documented first-time loading pipeline from the official CMS release files.

This document adapts the author's reference architecture (*Clinical Ontology Knowledge Graph — Reference Architecture for ICD-10-CM and Multi-Ontology Integration*, v1.0) to hdh's actual mechanisms: the schema registry, the core/modules separation, the design-quality gates, and the agent pipeline. Where this design deliberately departs from the reference, §2 says so and why — those departures are exactly what we want RFC feedback on.

Contents

1. Purpose and positioning

hdh's ontology module today is a scaffold: a starter ICD-10→SNOMED map and two registry-added columns on `Diagnosis`. Every other module wants more than that — care-gap rules reason about code families, the risk model buckets diagnoses by chapter, the care-plan design (see `care-plan-module.md`) needs SNOMED-coded health concerns, and the agent answers questions like "*how many forearm fractures last winter?*" that are really hierarchy roll-ups.

The insight this module carries over from the author's work on public-safety knowledge graphs: **agentic AI explores data dramatically better when the domain's relationships are materialized as a graph** rather than left implicit in string prefixes. ICD-10-CM is a natural fit — the code system *is* a hierarchy (Chapters → Blocks → Categories → Codes), and its positional semantics (laterality, episode of care) and coding rules (Excludes1/2, code-first, use-additional) are typed edges waiting to be extracted.

What this module is:

- The full ICD-10-CM catalog (~74k codes, FY release) as queryable entities with hierarchy, laterality, episode, and coding-rule edges
- A lookup service with honest latency engineering: sub-10ms hot-path code lookup, full-text code search, hierarchy roll-ups
- The project's first **multi-table schema-registry module** — new entities, not just appended columns — and the driver for registry v2 capabilities
- The vehicle for hdh's move to PostgreSQL + Redis (`just deps`) — and the **retirement of SQLite** as a supported dialect (§5.3)

What it is not. Not a certified coding product, not a terminology server (no CTS2/FHIR `$lookup` conformance claimed in v1), and not a replacement for the existing ontology module — `icd10cm` *depends on* `ontology_module` and links into its SNOMED columns.

Educational candor. At 74k rows the entire catalog fits in process memory, and PostgreSQL's own indexes and buffer cache already serve the latency targets in §7. The reference architecture's multi-tier cache (L1/L2/L3, warming, invalidation machinery) is therefore **deliberately not built** — §6 explains the restraint, and `hdh icd bench` exists to prove the targets are met without it (or to tell us honestly if they aren't). Knowing when *not* to add a tier is as much a design lesson as knowing how to build one.

2. Review of the reference architecture

The reference document gets the big things right, and this design adopts them wholesale:

Adopted	Why
Entities + typed relationship edges as the core model	Matches the law-enforcement graph pattern; makes coding rules and laterality queryable instead of implicit
Access-pattern-driven optimization (lookup dominates; explicit latency targets per pattern)	Correct workload analysis for coding workflows; §7 keeps the targets and adds a benchmark harness to verify them
Materialized path + recursive CTEs for hierarchy	Right tool on PostgreSQL; roll-ups become index-range scans
Cross-ontology mappings as first-class edges with authority + confidence	The only design that scales past two ontologies; §9 adopts the edge model
Ontology registry for pluggable loaders/parsers	Directly congruent with hdh's module philosophy

Where this design departs — each departure is an RFC discussion point (§12):

1. **SQLAlchemy 2.0 and one `Base`.** The reference code declares its own `declarative_base()` in 1.x style. hdh has a single `Base` and a schema registry whose whole job is letting modules add entities declaratively. The reference's illustrative code becomes JSON schema specs (§3); the reference remains the conceptual source, not the literal one.

2. **Laterality by description, not by character position.** The reference's `LATERALITY_POSITIONS` map keys laterality position off the code's first letter (`'S': 7`) — which contradicts its own §1.2 (in `S52.001A`, character **6** is laterality; 7 is episode) and breaks across chapters where laterality sits at character 5 or 6. Positional heuristics are brittle; the official long descriptions are not. The loader derives laterality by normalizing side tokens in the description ("…*of right ulna*" / "…*of left ulna*" → same normalized stem = one laterality group). This is data-driven, self-correcting across fiscal years, and testable against the whole catalog.

3. **Laterality groups as a key, not $O(n^2)$ edges.** The reference's builder emits all-pairs bidirectional edges per group ($n \cdot (n-1)$ rows). We store a `laterality_group` key on each member plus one `CONTRALATERAL` edge per left/right pair; variant listing is a keyed index scan. Its laterality materialized view also joins on `category_code` alone, which would pair `S52.001-` with `S52.5xx-` codes across different subcategories — the group key eliminates that class of bug.

4. **Prefix, not substring, path matching.** The reference's descendant query uses `LIKE '%S52%'` — a leading wildcard that defeats its own `text_pattern_ops` index and matches `S52` anywhere in the path. All path queries here are anchored prefix matches (`LIKE 'ch19.S50-S59.S52%'`).

5. **No Elasticsearch, no GraphQL, Claude not GPT-4.** PostgreSQL FTS (an expression GIN index over tsvector, with `pg_trgm` for fuzzy) is ample at this corpus size and removes an operational dependency. NL-to-query stays in the existing Anthropic-SDK agent pipeline — the module ships *tools*, not its own LLM stack (§8).

6. **No tiered cache.** The reference's largest single component — the L1/L2/L3 cache with warming, TTL policy per data type, and SCAN-and-delete invalidation — is dropped entirely, not fixed. At 74k concepts the working set fits in PostgreSQL's buffer cache, the hot path is a primary-key lookup, and the dataset changes once a year; the cache tiers add three failure modes (staleness, stampede, half-completed invalidation) to solve a latency problem we cannot measure yet. §6 states the restraint policy and the re-

entry criteria. Redis stays in the dependency stack, but as the **future Celery broker** for background jobs — not as a cache (§5).

7. **PostgreSQL-only — and the reference is right.** Early drafts of this design kept SQLite alive beside PostgreSQL: LIKE fallbacks for FTS, JSON1 branches for JSONB, executemany beside COPY, a capability matrix, a doubled CI. That is an appendix, not an architecture — every feature would ship twice and be tested twice, forever. This module is the moment hdh **retires SQLite entirely** and rewrites the laptop story around containers (§5): one dialect, one code path, real foreign keys under concurrency, JSONB, matviews, and honest migrations. The exit is engineered, not abrupt — `hdh migrate` carries every existing database across (§5.3).

3. Data model as a schema-registry module

3.1 Module layout

```
src/hdh/modules/icd10cm/
├── manifest.json                 {"name": "icd10cm_module",
│                                  "depends_on": ["base", "ontology_module"],
│                                  "priority": 20}
├── schema/
│   ├── entities/
│   │   ├── ontology_concept.json      NEW entity → ontology_concepts
│   │   ├── ontology_edge.json         NEW entity → ontology_edges
│   │   ├── ontology_load.json         NEW entity → ontology_loads
│   │   └── diagnosis.json             EXTENDS Diagnosis (+concept link)
│   └── relationships/
│       └── diagnosis_concept.json     Diagnosis.concept → OntologyConcept
├── ddl/postgresql/                    dialect-gated accelerators (§3.4)
│   ├── 01_search_index.sql
│   ├── 02_hierarchy_matview.sql
│   └── 03_trgm.sql
├── loader/                            §4 pipeline stages
├── service.py                         §7 typed lookup API
├── tools.py                           §8 agent tools
└── cli.py                             hdh icd …
```

The manifest's `depends_on: ["ontology_module"]` exercises the registry's topological ordering with a real dependency for the first time — icd10cm's relationship specs may reference columns the ontology module added.

3.2 Entities

Three new entities, declared as JSON specs and materialized by the registry's two-pass factory. The **generic-graph vs. typed-tables** tension (reference `ClinicalEntity` vs. hdh's typed ORM) resolves as: *generic storage, typed access*. Concepts and edges are deliberately ontology-agnostic tables — SNOMED/LOINC/CPT load into the same shape later — while the public API returns frozen dataclasses (§7), never raw rows, per the `data-abstraction` quality gate.

OntologyConcept (`ontology_concepts`) — one row per code/concept:

Column	Type	Notes
`id`	String(64) PK	`icd10cm:S52.001A` — ontology-qualified
`ontology`	String(16), indexed	`icd10cm` now; `snomed_ct`, `loinc` … later
`code`	String(32), indexed	the bare code
`kind`	Enum	`chapter` · `block` · `category` · `subcategory` · `code`
`display`	String(512)	long description
`short_display`	String(128)	short description (order file col 5)
`is_billable`	Boolean	order file "valid for submission" flag
`hierarchy_depth`	Integer	0 = chapter
`path`	String(1024), indexed	materialized path `ch19.S50-S59.S52.S52.0.S52.001`
`laterality`	String(1), nullable	`1` right · `2` left · `9` unspecified
`laterality_group`	String(64), nullable, indexed	normalized-description stem key (§2 pt 2–3)
`episode`	String(1), nullable	`A` initial · `D` subsequent · `S` sequela …
`episode_group`	String(64), nullable, indexed	code minus 7th character
`properties`	JSON	ontology-specific extras (JSONB on PostgreSQL)
`effective_fy`	Integer	first fiscal year present
`retired_fy`	Integer, nullable	set by addenda diffs — codes are never deleted

OntologyEdge (`ontology_edges`) — typed relationships:

Column	Type	Notes
`id`	Integer PK	
`source_id` / `target_id`	String(64) FK → concepts	
`edge_type`	Enum	`parent_of` · `contralateral` · `axis_variant` · `episode_variant` · `excludes1` · `excludes2` · `code_first` · `use_additional` · `includes` · `maps_to`
`authority`	String(64)	`CMS_TABULAR`, `DERIVED_LOADER`, `NLM_UMLS` …
`confidence`	Float, default 1.0	1.0 for official; <1.0 for probabilistic mappings
`properties`	JSON	e.g. Excludes note text

Composite indexes on `(source_id, edge_type)` and `(target_id, edge_type)`. Hierarchy stores **only** `parent_of` — ancestor/descendant closure comes from the path column and the matview, never from materialized transitive edges.

OntologyLoad (`ontology_loads`) — the load ledger (§4.4): one row per completed load with fiscal year, source file checksums, row/edge counts, and duration.

Diagnosis extension (`diagnosis.json`) — one new column `concept_id` (String(64), FK → `ontology_concepts.id`, nullable) plus a `concept` relationship. This is the bridge that makes every existing `Diagnosis` row graph-addressable; `hdh icd link` backfills it (§4.5), the same pattern as `hdh ontology tag`.

3.3 Registry v2: what the mechanism must grow

The current registry supports scalar column types, FKs, and relationships. This module needs four additions — each useful to every future module:

Addition	Spec form	Registry change
`JSON` column type	`{"type": "JSON"}`	map to `JSON().with_variant(JSONB, "postgresql")`
Composite / partial indexes	`"indexes": [{"name": …, "columns": […], "where": …}]` per entity spec	emit `Index` objects in factory pass 1
Server defaults	`"server_default": "now()"`	pass through to `Column`
Dialect-gated DDL hooks	`ddl/<dialect>/*.sql` in module dir	new phase after `create_all` / `ensure_columns` : execute in filename order when `engine.dialect.name` matches, recorded in `ontology_loads.properties` for idempotency

Core stays ignorant of modules: the DDL hook is generic ("run a module's dialect-matching DDL"), not ICD-specific.

3.4 PostgreSQL accelerators (dialect-gated DDL)

- `01_search_index.sql` — one **expression GIN index**: `GIN (to_tsvector('english', code || ' ' || display))` . Deliberately *not* the reference's stored `search_vector` column + maintenance trigger: an expression index gives identical query power (`@@ plainto_tsquery` , `ts_rank`) with zero maintained state. Stored vectors pay off on write-heavy tables; this one is written once a year by the loader.
- `02_hierarchy_matview.sql` — `icd10cm_hierarchy(code_id, ancestor_id, distance)` closure matview built with a bounded recursive CTE over `parent_of` edges; refreshed `CONCURRENTLY` at the end of each load.
- `03_trgm.sql` — `pg_trgm` extension + trigram index on `display` for fuzzy/misspelled term search.

With a single dialect these are not "accelerators with fallbacks" — they are simply how search and hierarchy work. The `ddl/<dialect>/` layout stays (it keeps the registry mechanism generic), but only `ddl/postgresql/` exists.

3.5 Semantic axes: what a code *means*, stored as graph

A code is not a string — it is a bundle of clinical decisions: which bone, which side, which aspect of the bone, how severe, which encounter. The representational strategy of this module is to make each of those **axes** explicit data, because retrieval (§7) works by matching axes, not by parsing code strings.

Worked example: ankle fracture. The malleolus family shows every axis at once — including the medial-vs-lateral distinction laterality alone cannot express:

```
S82.5  Fracture of MEDIAL malleolus (tibia)    S82.6  Fracture of LATERAL malleolus (fibula)
├─ S82.51  displaced,    right                 ├─ S82.61  displaced,    right
├─ S82.52  displaced,    left                  ├─ S82.62  displaced,    left
├─ S82.53  displaced,    unspecified           ├─ S82.63  displaced,    unspecified
├─ S82.54  nondisplaced, right                 ├─ S82.64  nondisplaced, right
├─ S82.55  nondisplaced, left                  ├─ S82.65  nondisplaced, left
└─ S82.56  nondisplaced, unspecified           └─ S82.66  nondisplaced, unspecified

   every leaf × 7th character:  A initial, closed · B initial, open
                                D routine healing · G delayed healing
                                K nonunion · P malunion · S sequela
```

Note where each axis actually lives — **no single positional rule covers them**:

Axis	Where ICD-10-CM encodes it	In this family
Anatomic aspect (medial/lateral)	the **category** split	S82.5 vs S82.6 — and it drags the bone with it (tibia vs fibula)
Severity (displaced/nondisplaced)	5th-character **banding**	.51–.53 vs .54–.56
Laterality (right/left/unspec)	6th character *within* each band	.x1/.x2/.x3 pattern
Encounter + healing course	7th character	A/B/D/G/K/P/S

This is exactly why §2's departure 2 rejects positional parsing: the same clinical axis surfaces at a different structural level in every code family. The **enrich** stage (§4.2) therefore extracts axes from the official long descriptions against a curated axis lexicon (side, aspect — medial/lateral/anterior/posterior — displacement, exposure open/closed, encounter), validates encounter values against the tabular XML's seventh-character definitions, and stores the result in `properties.axes` (JSONB, GIN-indexable on PostgreSQL).

The stored neighborhood for one node:

```
(icd10cm:S82.52  "Displaced fracture of medial malleolus of left tibia"
   axes: {bone: tibia, site: malleolus, aspect: medial,
          laterality: left, displacement: displaced})
    │
    ├─ contralateral ──────────────────▶ S82.51  (left ⇔ right)
    ├─ axis_variant {axis: displacement} ─▶ S82.55  (displaced ⇔ nondisplaced, same side)
    ├─ axis_variant {axis: aspect} ──────▶ S82.62  (medial ⇔ lateral, same side & severity)
    ├─ episode_variant ────────────────▶ S82.52XA · S82.52XD · S82.52XS …
    └─ child_of ─▶ S82.5 ─▶ S82 ─▶ S80-S89 ─▶ Chapter 19
```

Sibling edges (`contralateral`, `axis_variant`, `episode_variant`) are built by the loader **within normalized-description stem groups** — two codes are axis variants when their descriptions are identical after removing exactly one axis's tokens. One generic `axis_variant` edge type with `properties.axis` covers aspect, displacement, and future axes without growing the enum; laterality and episode keep their named types because they are the two the reference identified as first-class and the two consumers query most.

The payoff is that clinical navigation becomes graph traversal: "*same fracture, other side*" is one edge; "*the nondisplaced version*" is one edge; "*how severe can this get*" is the axis_variant fan-out; "*is this still billable if I*

drop laterality" is a `child_of` hop plus a flag check. §7.3 builds the query language on exactly these moves.

4. First-time loading of ICD-10-CM

The section the reference architecture lacked entirely: where the codes actually come from and how they get in.

4.1 Sources (all public domain)

File	From	Gives us
`icd10cm-order-FY.txt`	CMS annual release (e.g. FY2026)	every code in tabular order: code, billable flag, short + long description — the backbone (~74k codes + ~22k headers)
`icd10cm-tabular-FY.xml`	CMS	chapter/block structure, Includes/Excludes1/Excludes2 notes, code-first / use-additional instructions, seventh-character definitions
`icd10cm-addenda-FY.txt`	CMS	year-over-year deltas — drives non-destructive updates (§4.4)

ICD-10-CM is published by NCHS/CMS and is **not licensed** — it ships-free. The loader takes `--source <dir>` (files already downloaded) or `--download` (fetch from the CMS URL for the requested FY, verify size and checksum, cache under `~/.hdh/icd10cm/FY/`). CI uses a committed 200-code fixture slice, never the network.

4.2 Pipeline

Nine stages, each a small pluggable class (`LoadStage` protocol: `name`, `run(ctx) -> StageResult`), composed by the loader — same pattern as the quality gate's checks:

```
1 acquire      download or locate files; checksum; record provenance
2 parse        order file → CodeRow stream (code, billable, short, long)
3 structure    tabular XML → chapters, blocks; derive category/subcategory
               nesting from code prefixes; compute path + depth
4 enrich       semantic-axis extraction (§3.5):
                 · episode: 7th char of 7-char codes, validated against
                   the XML's seventh-character definitions
                 · laterality: normalize side tokens in long description
                   ("right"/"left"/"unspecified …") → laterality flag +
                   shared laterality_group stem   (§2, departure 2)
                 · other axes via the axis lexicon (aspect medial/lateral,
                   displacement, exposure, …) → properties.axes JSONB
5 load         bulk insert concepts in 5k batches
               (COPY via psycopg — single dialect, single fast path)
6 edges        parent_of from nesting; contralateral pairs within each
               laterality_group; episode_variant within episode_group;
               excludes1/2, code_first, use_additional from tabular notes
7 accelerate   run dialect DDL (§3.4); REFRESH MATERIALIZED VIEW
8 verify       invariant checks — every non-chapter has a parent; every
               7-char code has a valid episode; laterality groups contain
               ≤1 of each side; billable count within ±2% of published
               figure; spot-check golden codes (E11.9, S52.001A …)
9 finalize     write OntologyLoad row; ANALYZE the new tables
```

A failed stage aborts before `finalize`; the load is transactional per stage and resumable with `--resume` (stages are idempotent — 5 upserts on `id`, 6 rebuilds edges for the load's FY).

Expected first-load time (target, to be benchmarked): under 2 minutes on the reference laptop against the `just deps` container.

4.3 CLI

```
hdh icd load --download --fy 2026        # first-time load
hdh icd load --source ./cms-files --fy 2026
hdh icd status                           # loads ledger, counts, active FY
hdh icd lookup S52.001A                  # full context: hierarchy, laterality,
                                         #   episode variants, coding rules
hdh icd search "fracture forearm"        # FTS (fuzzy on PostgreSQL)
hdh icd lateral S52.001A                 # → S52.002A (contralateral)
hdh icd link                             # backfill Diagnosis.concept_id
hdh icd bench                            # measure §7's latency targets
```

4.4 Updates without destruction

Fiscal-year updates (annual, October) load via the addenda: added codes insert with `effective_fy`, deleted codes set `retired_fy` (rows never deleted — historical `Diagnosis` rows keep valid FKs), revised descriptions update in place with the old value archived to `properties.history`. Each update is a new `OntologyLoad` row. Full reload is always available and always safe for the same reason.

4.5 Linking the synthetic EHR

`hdh icd link` matches `Diagnosis.icd10_code` to concepts (the generator's 30+ profiles use real codes, so match rate should be ~100%) and sets `concept_id`. From that moment every hierarchy/laterality/rules query composes with patient data — the payoff the agent tools cash in (§8).

5. PostgreSQL migration and dependency containers

5.1 `just deps`

A new `docker-compose.deps.yml` at the repo root:

```
services:
  postgres:
    image: postgres:16-alpine
    environment: {POSTGRES_USER: hdh, POSTGRES_PASSWORD: hdh, POSTGRES_DB: hdh}
    ports: ["5432:5432"]
    volumes: [hdh-pgdata:/var/lib/postgresql/data]
    healthcheck: {test: ["CMD-SHELL", "pg_isready -U hdh"], interval: 2s, retries: 15}
  redis:
    image: redis:7-alpine
    ports: ["6379:6379"]
    healthcheck: {test: ["CMD", "redis-cli", "ping"], interval: 2s, retries: 15}
volumes:
  hdh-pgdata:
```

```
# justfile additions
deps:          # start PostgreSQL + Redis and wait until healthy
    docker compose -f docker-compose.deps.yml up -d --wait
deps-down:     # stop dependency containers (data volume preserved)
    docker compose -f docker-compose.deps.yml down
deps-nuke:     # stop and DELETE the data volume
    docker compose -f docker-compose.deps.yml down -v
```

`.env.example` gains (values are local-container defaults, not secrets):

```
HDH_DB_URL=postgresql+psycopg://hdh:hdh@localhost:5432/hdh
HDH_REDIS_URL=redis://localhost:6379/0   # reserved: Celery broker/beat (§6)
```

Redis ships in the stack from day one so the compose file never has to change, but nothing in this module talks to it in v1 — it is reserved for the Celery worker/beat phase (§6, "Redis's actual job").

5.2 The rewritten laptop story

The old promise was "runs on a bare laptop with nothing installed." The new promise is **"runs on any laptop with a container runtime"** — which in 2026 is every developer laptop:

Platform	Prerequisite	Then
macOS	Docker Desktop (or OrbStack/colima)	`just deps`
Linux	docker engine + compose plugin	`just deps`
Windows	Docker Desktop with WSL2 backend	`just deps`

One command, ~30 seconds, done — the compose file's `--wait` healthchecks mean `just deps` returns only when the database accepts connections. The practitioner guide gets a one-page "install Docker Desktop" section with screenshots; that is the entire cost of the new story. `just check-env` extends to verify DB connectivity and prints "run `just deps`" when the container is down — the first-run failure mode is a friendly sentence, not a stack trace.

`get_engine()` reads `HDH_DB_URL` (default in `.env.example` points at the `just deps` container) and builds a QueuePool engine; `psycopg[binary]` moves into core dependencies. What hdh buys with the one prerequisite:

real foreign-key enforcement under concurrent writers, JSONB + GIN, matviews, recursive CTEs, COPY bulk loading, one code path per feature, and a test suite that tests exactly what production runs.

5.3 Migration away from SQLite — the exit path

SQLite support is not dropped; it is **retired on a schedule**, so it never lingers as a second dialect:

1. `hdh migrate` (ships in Phase 1): copies an existing `family_medicine.db` into PostgreSQL — metadata-driven table walk, FK-ordered batched inserts, verified by row counts and spot-check checksums. One command, idempotent, and the SQLite file is left untouched as its own backup.
2. `hdh generate` **writes to** `HDH_DB_URL` — new datasets are born in PostgreSQL. The pre-built release asset becomes a `pg_dump` custom archive (`just db-restore family_medicine-10k.pgdump`); the SQLite zip ships beside it for exactly one more release.
3. **SQLite read support survives only inside** `hdh migrate` — the one place the sqlite dialect remains reachable. Core, modules, tests, and CI drop every SQLite branch: `ensure_columns()` 's ALTER shim retires in favor of **Alembic (issue #7 closes)** as the single migration mechanism, with the schema registry emitting migration operations.
4. **CI runs one configuration**: GitHub Actions service containers for postgres + redis — the same images `just deps` starts. The container-free job disappears along with the code it existed to test.

The deleted-code ledger *is* the robustness argument: no LIKE search fallback, no JSON1 branch, no executemany path, no dialect capability matrix, no doubled CI — and every remaining line runs in production configuration every time.

6. Caching: deliberately minimal

The reference architecture's most elaborate component is its multi-tier cache. This design's position: **at this scale, that is the wrong thing to build.** The removal is a design decision with reasons, not an omission:

- **The database already caches.** The hot path (`lookup`) is a primary-key fetch on a 74k-row table; PostgreSQL's buffer cache keeps the whole table resident after first touch. Sub-10ms is the *default* here, not an achievement.
- **The data changes once a year.** A cache's costs are staleness, stampede, and invalidation bugs; its benefit is avoided recomputation. A read-only annual dataset offers almost no recomputation to avoid.
- **Premature tiers hide real numbers.** `hdh icd bench` measures every §7 access pattern against the bare database first. Optimization that precedes measurement is speculation.

What remains is the minimum with obvious wins and no failure modes:

Kept	Form
Hierarchy closure	PostgreSQL materialized view (§3.4) — precomputation in the database, refreshed by the loader, no invalidation logic in Python
Per-process memoization	a bounded `functools.lru_cache` on `full_context()` only — evaporates with the process, cannot go stale within one

Re-entry criteria. A shared cache earns its way back in only when `hdh icd bench` shows a §7 target missed under a real multi-worker load (e.g., the FHIR API serving concurrent `full_context` calls), and the fix is applied to that measured path only. The RFC asks reviewers to challenge this restraint (§12 Q4).

Redis's actual job. Redis stays in the `just deps` stack (§5.1) — not as a cache, but reserved as the **broker/result backend for Celery workers and beat** in a later phase: async `hdh icd load --download` jobs, a scheduled fiscal-year addenda check (beat, annually around October 1), matview refresh after load, and eventually agent-pipeline background tasks. Job queues are Redis's earning use case here; caching was not.

7. Retrieval: lookup, description-to-code, and graph queries

Storage is only half the module; this section is the other half — three retrieval strategies layered from cheapest to richest: direct operations (§7.1), the description→code funnel (§7.2), and LLM-generated graph patterns compiled to SQL (§7.3).

7.1 Direct operations

`service.py` exposes a typed facade; all returns are frozen dataclasses (`Concept`, `CodeContext`, `LateralityGroup`, `SearchHit`, `CodedCandidate`), constructed from rows at the boundary:

Operation	Access pattern share (ref. §3.1)	Target	Strategy
`lookup(code)`	~60%	< 10ms	PK on qualified id
`search(term)`	~25%	< 50ms	expression-GIN FTS + trigram fuzzy
`ancestors(code)` / `descendants(code)`	~10%	< 20ms	path prefix / closure matview
`lateral_variants(code)` / `contralateral(code)`	~8%	< 10ms	`laterality_group` index
`coding_rules(code)`	—	< 20ms	`(source_id, edge_type)` index
`crosswalk(code, target_ontology)`	~5%	< 50ms	`maps_to` edges
`full_context(code)`	composite	< 100ms	one call the agent tools use: concept + ancestors + laterality + episode variants + rules + mappings
`codify(description)`	composite	< 500ms + LLM	the §7.2 funnel → ranked `CodedCandidate` list with explanation paths

`descendants` supports `billable_only=True` (the "all billable codes under S52" pattern) via a partial index on PostgreSQL. Every operation takes an injected `Session` — no globals, no construction. `full_context()` carries the one `lru_cache` memo (§6).

7.2 Description → code: the retrieval funnel

The flagship consumer question — *"what is the ICD-10-CM code for …?"* — is answered by a four-stage funnel in which **the LLM only ever classifies; deterministic code does all retrieval**:

```
user description — "elderly woman slipped, broke the inner side of her
                    left ankle, first visit, skin intact"
        │
1 AXIS EXTRACTION      LLM, schema-enforced structured output (the
        │              VERDICT_SCHEMA discipline): {site: "ankle/malleolus",
        │              aspect: medial, laterality: left, encounter:
        │              initial, exposure: closed, mechanism: fall}
        ▼
2 CANDIDATE ANCHORING  deterministic: FTS (plainto_tsquery over
        │              canonical terms from step 1) + trigram fallback
        │              for misspellings → candidate categories (S82.5…)
        ▼
3 GRAPH DESCENT        deterministic: walk child_of from each anchor,
        │              filter properties.axes ⊇ extracted axes,
        │              billable only, apply excludes1/code_first edges
        ▼
4 RANKING              axis-match count → ts_rank → specificity depth;
                       top-k CodedCandidates, each carrying its hierarchy
                       path and per-axis match table as the explanation
```

Result here: **S82.52XA** (*Displaced fracture of medial malleolus of left tibia, initial encounter for closed fracture*) ranked first, with S82.55XA (nondisplaced) and S82.62XA (lateral) as visible, one-edge-away alternates — because §3.5 stored them as neighbors, the funnel can show *why* it chose and what the near-misses were. "Inner side of ankle" → `aspect: medial` is the LLM's classification job; everything after step 1 is reproducible SQL.

Two candor notes. Displacement was never stated, so the funnel must either rank the displaced/nondisplaced variants equal and ask, or prefer the unspecified parent — surfacing *missing axes as questions* is a feature (the agent can ask "was the fracture displaced?"), and §12 Q9 asks reviewers where to draw that line. And this is an educational coding assistant over synthetic data — not billing advice; suggestions carry their explanation paths precisely so a human can check them.

7.3 Graph patterns from the LLM, SQL from the compiler

For questions beyond single-code resolution (*"all left-sided billable fracture codes with a nonunion episode"*), the module adopts the reference's NL→graph-pattern idea with one hard rule: **the LLM emits a closed JSON pattern, never SQL.** A deterministic compiler turns the pattern into parameterized SQL — the `injection-safety` quality gate is satisfied by construction, and every query is explainable and replayable.

Pattern (structured output against `PATTERN_SCHEMA`; every field enumerated, no free text reaches SQL):

```
{
  "anchor":     {"terms": "fracture malleolus"},
  "axes":       {"aspect": "medial", "laterality": "left",
                 "encounter": "initial", "exposure": "closed"},
  "traverse":   [{"edge": "parent_of", "dir": "down", "depth": "*"}],
  "constraints": {"billable": true},
  "rank":       ["axis_match", "text_rank", "specificity"]
}
```

Compilation rules (each pattern element has exactly one SQL form):

Pattern element	Compiles to
`anchor.terms`	FTS predicate on the §3.4 expression index (bound parameter)
`traverse parent_of, dir down, depth *`	anchored path-prefix `LIKE` (§2, departure 4) — the graph hop *is* the path index
`traverse <typed edge>, depth 1`	self-join on `ontology_edges (source_id, edge_type)`
`traverse depth 2..n`	bounded recursive CTE (never unbounded)
`axes`	JSONB containment `properties->'axes' @> :axes` (GIN-indexed)
`constraints` / `rank`	WHERE flags / deterministic ORDER BY expression

Compiled sketch for the pattern above:

```
WITH anchor AS (
  SELECT id, path FROM ontology_concepts
  WHERE ontology = 'icd10cm'
    AND to_tsvector('english', code || ' ' || display)
        @@ plainto_tsquery('english', :terms)
),
candidates AS (
  SELECT c.* FROM ontology_concepts c
  JOIN anchor a ON c.path LIKE a.path || '%'        -- parent_of* descent
  WHERE c.is_billable AND c.kind = 'code'
)
SELECT c.code, c.display,
       (c.properties->'axes' @> :axes_full)::int          AS full_match,
       hdh_axis_overlap(c.properties->'axes', :axes_full) AS axis_score
FROM candidates c
ORDER BY full_match DESC, axis_score DESC,
         ts_rank(to_tsvector('english', c.display),
                 plainto_tsquery('english', :terms)) DESC,
         c.hierarchy_depth DESC
LIMIT :k;
```

The compiler is ~200 lines of boring, fully unit-testable Python, and it is the module's answer to the reference's `ClinicalQueryPatternGenerator` (which prompted the LLM for a pattern *and trusted it to be right*): here an invalid pattern fails schema validation and returns to the LLM with feedback — the same retry-with-feedback loop the agent pipeline already uses for response validation.

7.4 Considered alternative: semantic (vector) anchoring

The funnel's most failure-prone stage is candidate anchoring (§7.2 step 2). Lexical FTS anchors only when the user's words share stems with the official descriptions — but people say *"heart attack"*, *"broken collarbone"*, *"inner ankle bone"*; the catalog says *myocardial infarction*, *fracture of clavicle*, *medial malleolus*. Trigram fixes spelling, not synonymy. When anchoring returns nothing useful, the agent retries with reformulated terms — it usually gets there, but every round costs tokens and latency.

The candidate fix from the RAG world — a sentence-transformer embedding per code plus nearest-neighbor search — is credible here, with four qualifications this design would insist on:

1. **pgvector, not a vector database.** At ~74k codes × 768 dims (~230 MB, one HNSW index), the `pgvector` extension drops into the same postgres:16 container `just deps` already runs. A standalone vector store at this scale would repeat the cache-tier mistake §6 just corrected.
2. **A clinical encoder, not a generic one.** MiniLM-class models underperform on medical vocabulary; SapBERT-class encoders (self-aligned on UMLS synonym pairs) are built for exactly the lay-term → formal-term bridge, run locally on CPU, and would ship as an optional `[semantic]` extra with the model version recorded in `ontology_loads`.
3. **Hybrid fusion, axes untouched.** Encoders are notoriously weak at the attributes our graph is strong at — *left* vs *right*, *displaced* vs *nondisplaced* embed nearly identically. Vector-only anchoring would raise recall and quietly wreck laterality precision. The shape is FTS + KNN fused (reciprocal rank fusion), feeding the unchanged axis filter and graph descent in steps 3–4.
4. **The deterministic competitor goes first.** The CMS release includes the **alphabetic Index** (`icd10cm-index-FY.xml`) — the ontology's own synonym table, curated for decades for precisely this human-phrase→code problem. Loading index terms as searchable synonym entities is a loader stage, not an ML system, and likely closes much of the vocabulary gap alone.

Decision procedure, not a decision. Same discipline as §6: the trace DB already counts anchor-misses and retry rounds per run, and the golden-description set (§10) is the corpus. Run it lexical-only, then +index-terms, then +hybrid-vector; each layer must earn its keep on measured retries before it ships. §12 Q10 puts the question to reviewers. (The care-plan module's KnowledgeStore left "BM25 default, vector optional" open — pgvector would be one shared answer for both modules.)

8. Consumers: the hdh agent and the care-plan module

8.1 Agent tools — including code-from-description for any user

The module registers agent tools (same discovery as CLI registration), and the intent analyzer gains a `coding` intent mapped to them in `INTENT_TOOLS`, keeping the token-economy discipline (only these tool schemas load for coding questions):

Tool	Answers
`icd_codify`	"what's the code for a broken inner left ankle, first visit?" → the §7.2 funnel, candidates with explanation paths
`icd_lookup`	"What is S52.001A?" → `full_context`, clipped through the existing `clip_tool_results`
`icd_search`	"code for abrasion of scalp?"
`icd_hierarchy`	"all billable codes under S52" — then composes with `query_database` for "...and which patients have them?"
`icd_lateral`	"left-side equivalent of S52.001A"
`icd_rules`	"what codes are excluded with E11.9?"
`icd_pattern`	complex traversals via §7.3 — pattern proposed by the model, compiled and executed deterministically

This makes coding help a first-class `hdh agent` capability for **any user**, not just developers: describe the condition in plain words, get ranked candidates with the reasoning visible. The existing pipeline guarantees

compose for free — the **validator** confirms any code the answer cites actually came from tool results (a hallucinated code fails grounding and triggers the retry loop), and when the funnel reports a missing axis (§7.2), the agent's natural move is to ask the follow-up question ("was the fracture displaced?") rather than guess.

The demonstrable win: "*How many left-vs-right forearm fractures last winter, and were any coded with an Excludes1 conflict?*" — a question that requires hierarchy + laterality + rules + patient data in one traced, validated pipeline run. That becomes the flagship demo and a test case.

8.2 Care-plan codification

The care-plan design (`care-plan-module.md` §5) defines `HealthConcern`, `PlanGoal`, and `PlanIntervention` entities that must carry standard codings to export as a conformant FHIR `CarePlan`. This module is where those codings come from:

Care-plan element	FHIR target	Codification path
Health concerns	`Condition` (category `health-concern`)	`codify()` over the concern statement → ICD-10-CM concept + its `maps_to` SNOMED edge (the MCC eCare Plan IG expects both codings)
Goals	`Goal.target.measure`	LOINC — future loader, same concept/edge tables (§9)
Interventions	`ServiceRequest` / `MedicationRequest`	CPT/HCPCS edges when a licensed/public source is loaded (§9); until then, text + SNOMED starter map
Rule checking	plan-level validation	`excludes1` edges across a plan's concern set — two mutually-exclusive codes on one plan is a structural error the care-plan validator can catch deterministically

The care-plan subagent calls the same `codify()` service the chat agent uses — one retrieval implementation, two consumers. This is the concrete answer to the care-plan RFC's open question about where standards-based codification comes from: it comes from here.

9. Cross-ontology roadmap and licensing

The edge model (`maps_to`, authority, confidence) is ontology-agnostic by construction, but *shipping* mappings is a licensing question, and the RFC should be honest about it:

Ontology	License reality	Plan
ICD-10-CM	public domain	full catalog, this module
SNOMED CT	UMLS license (free for US affiliates, not redistributable)	keep shipping our starter map as `maps_to` edges (authority `HDH_STARTER`); document how a licensed user loads the full NLM ICD-10-CM↔SNOMED map themselves — loader stage, not data
LOINC	free with registration, redistribution restricted	same pattern: loader provided, data user-supplied
CPT	AMA-copyrighted, **paid**	schema supports it; hdh will never ship it
ICD-10-PCS / HCPCS	public domain	future loaders, same `LoadStage` pipeline

Note: CMS's ICD-10 GEMs crosswalks were last published for FY2019 and are frozen; they are usable as a historical demonstration corpus but should be labeled `authority=CMS_GEMS_2019, confidence<1.0`.

10. Testing and quality gates

- **Fixture slice** — ~200 codes committed (S52 family complete, E11 family, one full chapter skeleton, known Excludes1 pairs): every loader stage tested against an ephemeral PostgreSQL schema (local: `just deps`
- a per-run test database; CI: service containers). Pure parsing/enrichment stages stay DB-free unit tests.
- **Property tests over the full catalog** (post-load `verify` stage doubles as a pytest marked `@pytest.mark.fullload`): laterality groups well-formed, hierarchy acyclic and single-rooted per chapter, path ↔ parent_of agree.
- **Golden-code contract tests** — E11.9, S52.001A, M25.511, G89.4 assert exact expected context (the reference's appendix queries become tests).
- **Golden-description funnel tests** — ~50 curated plain-language descriptions with expected top candidates (the §7.2 ankle case among them), run offline with fixture axis-extraction JSON standing in for the LLM — stages 2–4 of the funnel are deterministic, so they test exactly.
- **Pattern-compiler tests** — every §7.3 pattern element to its SQL form, plus rejection tests: patterns with unknown fields, unbounded depth, or free-text leakage into SQL must fail schema validation.
- **Benchmark harness** (`hdh icd bench`) — asserts nothing in CI, prints the measured table for the README; targets in §7 are goals, measurements are truth.
- **Quality gate** — stages are pluggable protocol implementations; services take injected sessions; public API returns dataclasses; composition roots list grows by `icd10cm/cli.py`. The gate runs unchanged.

11. Phased implementation plan

Phase	Delivers	Proves
1. `just deps` + exit from SQLite	compose file, `HDH_DB_URL` engine path, `hdh migrate`, `just db-restore`, Alembic baseline (closes issue #7), CI on service containers, practitioner-guide Docker section	hdh runs — and stays — on PostgreSQL
2. Registry v2 + schema	JSON type, indexes, server defaults, DDL hooks; the three entities + Diagnosis link	multi-table schema modules work
3. Loader + CLI	all nine stages incl. COPY + accelerator DDL; fixture tests; `hdh icd load/lookup/search/lateral/link/status`	full catalog loads and queries
4. Bench + demo	`hdh icd bench` latency harness; measured §7 table in README	targets are met without a cache tier — or we learn where they are not
5. Retrieval + agent	axis lexicon + `codify()` funnel + pattern compiler; 7 tools + `coding` intent; flagship demo test	description→code for end users; graph + EHR + agent composition
6. Cross-ontology	SNOMED starter edges migrated to `maps_to`; licensed-map loader docs	the roadmap is real

Each phase is independently mergeable and demo-able; the RFC stays open through at least Phase 3.

12. Open questions (RFC prompts)

1. **Generic graph vs. typed tables.** Concepts/edges are deliberately ontology-agnostic (one table pair for all future ontologies) with typed dataclasses only at the API boundary. Would you push typing down into per-ontology tables (the reference's `ICD10CMCode` side-table), and what does that cost when ontology #3 arrives?
2. **Laterality by description normalization** — the side-token stemming approach replaces positional maps. What edge cases break it? (Bilateral codes, "unspecified eye," transplant status codes…) Is a hybrid — description-derived, position-validated — worth the complexity?
3. **Is retiring SQLite too aggressive?** The new baseline is "any laptop with a container runtime" (§5.2) — macOS, Linux, or Windows with Docker Desktop/WSL2. Does that exclude anyone hdh should care about (locked-down hospital workstations? teaching labs?), and is one transitional release with `hdh migrate` + the legacy SQLite zip a long enough exit ramp?
4. **Is the no-cache position right?** §6 drops the reference's tiered cache entirely and sets measured re-entry criteria. If you think a shared cache belongs in v1 after all — say for multi-worker FHIR API deployments — make the case with the workload that needs it.
5. **Celery scope.** Redis is reserved as a future Celery broker (§6): async loads, an annual beat check for new fiscal-year files, matview refresh. Is a job queue warranted for work this infrequent, or is a synchronous CLI (plus documentation) the honest v1?
6. **Alembic from day one** (§5.3): with a single dialect the registry can emit migration operations cleanly — does anything still argue for keeping the `ensure_columns` shim during the transition, or should the Alembic baseline land in Phase 1 as proposed?

7. **Structural critique** — the departures in §2 are arguments, not verdicts. If you'd have kept any reference decision we dropped (Elasticsearch, all-pairs laterality edges, positional laterality), make the case.
8. **Axis lexicon coverage.** Medial/lateral/displaced works for the injury chapters; ophthalmology, otology, obstetrics, and neoplasm families each carry their own axis vocabularies (behavior, trimester, stage…). Per-chapter lexicons, one grand lexicon, or derive candidate axes statistically from description diffs within stem groups — and who curates the result?
9. **Missing-axis policy.** When a description omits an axis the code requires (displacement unstated in the §7.2 example): ask a follow-up, default to the unspecified/parent code, or present both branches ranked equal? Coders, what does real workflow want?
10. **Semantic anchoring (§7.4).** For closing the lay-term → formal-term gap in candidate anchoring: official Index terms, a SapBERT-class encoder over pgvector, both layered, or neither? Especially interested in evidence on hybrid (RRF) retrieval for clinical concept normalization, and on encoders' laterality blindness — does anything change the "axes stay deterministic" conclusion?

References

- *Clinical Ontology Knowledge Graph — Reference Architecture for ICD-10-CM and Multi-Ontology Integration*, v1.0, August 2026 — the author's design notes this RFC adapts; not distributed with the repository.
- CMS, *ICD-10-CM Files* — annual order file, tabular XML, addenda (public domain).
- hdh, `docs/design/original-design-notes.md` §7–§13 — the schema-registry design this module extends.
- hdh, `docs/design/care-plan-module.md` — consumer of the SNOMED-coded concern entities this module strengthens.
- NLM UMLS licensing terms (SNOMED CT / LOINC redistribution constraints).